

程设实验项目二：用 EasyX 实现计算器

卢耀琪 251275016

2026 年 5 月 9 日

1 分析 EasyX 的函数使用

1.1 文字输出部分

1.1.1 文字样式

```
1 // standard.cpp — 大号粗体微软雅黑，用于显示计算结果
2 settextstyle(60, 0, _T("微软雅黑"), 0, 0, 900, false, false, false,
3     ...);
4
5 // standard.cpp — 普通大小，用于按钮上的数字
6 settextstyle(30, 0, _T("微软雅黑"), 0, 0, 900, 0, 0, 0);
7
8 // standard.cpp — 等宽字体 Consolas，用于按钮标签
9 settextstyle(23, 0, _T("Consolas"));
10
11 // help.cpp — 小号楷体，用于帮助文档
12 settextstyle(18, 0, _T("楷体"), 0, 0, 200, false, false, false);
13
14 // science.cpp — 用不同大小显示不同按钮标签
15 settextstyle(22, 0, _T("Consolas")); // 小字：sqrt、ln、log
16 settextstyle(30, 0, _T("Consolas")); // 中字：数字、C、=
17 settextstyle(40, 0, _T("Consolas")); // 大字：+、-、×、÷
```

- settextstyle 是 EasyX 图形库里用来设置文字样式的函数，用来控制：字体大小、字体名称、是否加粗 / 斜体、文字方向等。
- 其完整参数包括高度, 宽度, 字体名称, 文字方向, 字符方向, 粗细, 斜体, 下划线, 删除线。不启用某个参数效果则填 false。

1.1.2 文字颜色

```
1 // standard.cpp — 白色文字
2 settextcolor(RGB(255, 255, 255));
3
4 // set.cpp — 青色文字 (鼠标悬停时)
5 settextcolor(RGB(0, 255, 255));
6
7 // conversion.cpp — HSL 生成的青蓝色
8 settextcolor(HSLtoRGB(190, 1, 0.5));
9
10 // set.cpp — 白色
11 settextcolor(WHITE);
```

- settextcolor 专门设置文字颜色
- RGB (红, 绿, 蓝), 每个数字范围: 0 ~ 255。如 RGB(0, 255, 255), 红: 0 (没有红色), 绿: 255 (绿色拉满), 蓝: 255 (蓝色拉满)。
- HSLtoRGB 是一个颜色转换工具, 把 HSL 颜色 → 转换成电脑能识别的 RGB 颜色, 其中 H 代表颜色, S 代表饱和度, L 代表亮度。

1.1.3 写出文字

```
1 // standard.cpp — 在按钮位置输出数字
2 outtextxy(40, 350, 'C'); // C键
3 outtextxy(142, 560, '.'); // 小数点
4 outtextxy(40, 508, '1'); // 数字1
5 outtextxy(142, 508, '2'); // 数字2
6 outtextxy(244, 508, '3'); // 数字3
7
8 // standard.cpp — 输出运算符
9 outtextxy(342, 500, '+'); // 加号
10 outtextxy(342, 344, _T("×")); // 乘号
11 outtextxy(342, 396, _T("÷")); // 除号
12
13 // set.cpp — 侧边栏功能名称
14 outtextxy(20, 85 + fly1, _T("标准"));
15 outtextxy(20, 125 + fly1, _T("科学"));
16 outtextxy(20, 165 + fly1, _T("大数运算"));
17
```

```

18 // standard.cpp — 在矩形区域内右对齐显示"0"
19 RECT r1 = { 10, 200, 380, 300 }; // 矩形区域: 左10 上200 右380 下
    300
20 drawtext('0', &r1, DT_RIGHT | DT_VCENTER | DT_SINGLELINE |
    DT_END_ELLIPSIS | DT_MODIFYSTRING);

```

- outtextxy(x,y,"文字") 用于输出文字，坐标是文字左上角的位置，越往右 x 越大，越往下 y 越大，文字要用双引号括起来
- 但 outtextxy 只能在一个点写字，不能换行、不能自动排版。而 drawtext 可以在一个矩形区域里写字，支持换行、居中、左对齐、右对齐、自动换行。drawtext("文字", & 矩形, 格式)
- 表格总结如下：

表 1: drawtext 常用标志含义

标志	含义
DT_RIGHT	右对齐
DT_LEFT	左对齐
DT_VCENTER	垂直居中
DT_SINGLELINE	单行显示（不换行）
DT_WORDBREAK	自动换行
DT_END_ELLIPSIS	文字太长时末尾显示"..."
DT_MODIFYSTRING	配合 END_ELLIPSIS 使用，截断字符串

和 outtextxy 的区别：

表 2: outtextxy 与 drawtext 对比

	outtextxy	drawtext
定位方式	指定左上角坐标	指定一个矩形区域
对齐方式	无，只能从坐标开始写	支持左/右/居中对齐
换行	不支持	支持 DT_WORDBREAK
截断	不支持	支持 DT_END_ELLIPSIS
用途	按钮标签、固定位置文字	计算结果、长文本

1.1.4 设置文字背景透明

```
1 setbkmode() — standard.cpp
```

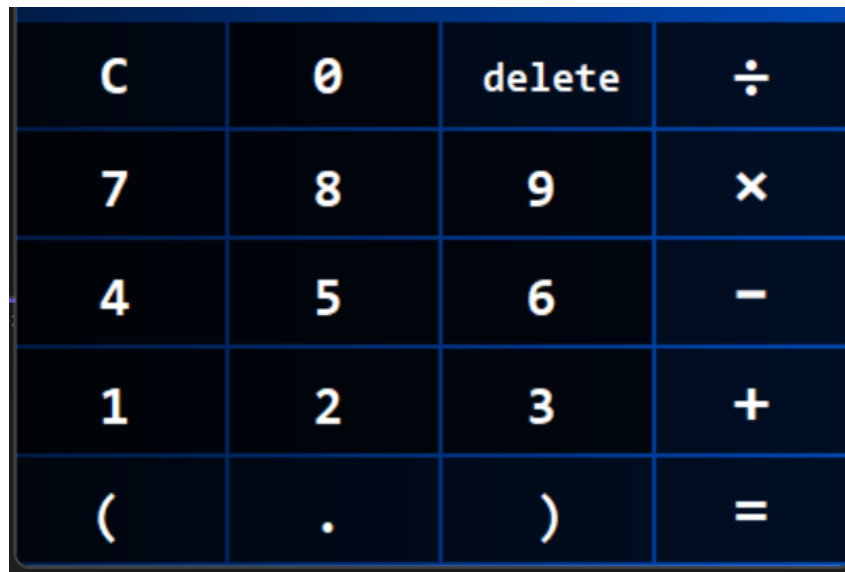
- setbkmode() 用于设置文字的背景模式。TRANSPARENT 是文字背景透明，而 OPAQUE 是文字背景不透明。
- 这个程序中所有界面都用透明模式，否则文字后面会有白色方块，setbkmode(TRANSPARENT);
- 这个函数在 EasyX 中很关键。如果不设 TRANSPARENT，每个文字后面都会带一个背景色方块，非常难看。作者在每个 showscreen 函数中都设置了它。

1.2 绘图

1.2.1 画直线

```
1 // conversion.cpp — changecolor函数中画渐变背景
2 for (int y = Left; y < Right; y++)
3 {
4     L += 1;
5     setlinecolor(HSLtoRGB(H, S, L)); // 每列颜色略变
6     line(y, Top, y, Bottom);         // 画一条竖线
7 }
8 // 循环400次，每条竖线颜色不同，合起来就是渐变色背景
9
10 // set.cpp — 画按钮的上下边框线
11 line(0, 40*(i-50), 299, 40*(i-50)); // 上边线
12 line(0, 40+40*(i-50), 299, 40+40*(i-50)); // 下边线
13
14 // set.cpp — 画侧边栏选中项的左侧竖线标记
15 for (int j = 0; j < 6; j++)
16 {
17     setlinecolor(HSLtoRGB(60, 1, 0.5));
18     line(j, 40*software[2]+5+fly1, j, 35+40*software[2]+fly1);
19 }
20 // 画6条紧挨着的竖线，形成一个粗的黄色竖条标记
```

- 如图所示，产生了渐变效果（截图可能看不清）



- L 是 HSL 颜色里的亮度（0 = 全黑，1 = 全白），每次增加 1 就是增加一点点，使其达到渐变的效果。

1.2.2 实心矩形

```
1 // loadimage.cpp - 填充整个IMAGE为纯色
2 solidrectangle(0, 0, width, height);
3
4 // set.cpp — 画选中功能的高亮背景条
5 solidrectangle(0, 40*(i-50), 299, 40+40*(i-50));
6
7 // conversion.cpp — 进制转换中高亮当前选中的进制
8 solidrectangle(0, fly1+47+40*i, 400, fly1+85+40*i);
```

- 用于画一个填满的矩形，但没有边框线，就是一整个色块

- 利用画线函数 `line`，当用户鼠标移动到不同的计算器选项时，会在其上下画出两条直线，然后使用 `solidrectangle` 函数在两行间画出一个无边框的矩形。这两个函数在使用时需要将对应的两个点的 (x, y) 坐标作为参数传给对应的函数（线给出的是起点和终点的坐标，矩形给出的是左上点以及右下点的坐标）



1.2.3 画边框

```
1 // help.cpp — "关于"弹窗的边框
2 setlinecolor(RGB(0, 238, 238)); // 青色
3 rectangle(0, 260, 350, 560);
4
5 // SolveSolution.cpp — 解方程帮助弹窗的边框
6 setlinecolor(RGB(0, 238, 238));
7 rectangle(210, 50, 390, 180);
8
9 // standard.cpp — "设置"文字的外框
10 setlinecolor(RGB(0, 238, 238));
11 rectangle(315, 45, 355, 65);
```

- 用当前线条颜色画一个空心矩形。只有四条边，不填充。

1.3 鼠标输入

```
1
2  MOUSEMSG m;           // 1. 声明鼠标消息变量
3  while (MouseHit())     // 2. 非阻塞检查是否有鼠标消息
4  {
5      m = GetMessage();  // 3. 取出一条消息
6  }
```

- MouseHit 用于判断是否有鼠标信息，主要包括鼠标的左键单击。GetMessage 用于获取鼠标信息，用于进行下一步的判断。是实现用户交互的重要函数。这个函数在新标准下已经废弃，只存在于 graphics.h 头文件中，可以利用 GetMessage 函数替代这个函数

1.4 双缓冲

```
1  void showscreen_()
2  {
3      BeginBatchDraw();    // 开始：告诉 EasyX 先别往屏幕上画
4
5      // ... 所有绘制操作 ...
6      // 全部画到内存缓冲区
7
8      EndBatchDraw();      // 结束：把内存缓冲区一次性复制到屏幕
9  }
```

- BeginBatchDraw() 之后的所有绘图操作都画到内存，用户完全看不到中间过程。EndBatchDraw() 调用时才真正刷新到显示器——所以用户看到的是完整的画面瞬间出现，没有任何闪烁。
- 与单缓冲（画背景 → 显示 → 画按钮 → 显示 → 画文字 → 显示用户看到：背景闪一下 → 按钮闪一下 → 文字出现）相比，双缓冲不会看到闪烁的过程，用户体验更好。

2 总结常用图形用户界面函数的画法

2.1 窗口初始化

```
1  initgraph(400, 600);          // 创建 400×600 窗口，程序入口
2  GetHWnd();                    // 获取窗口句柄
3  SetWindowText(GetHWnd(), _T("计算器"));    // 设置窗口标题
```

2.2 双缓冲

```
1  BeginBatchDraw();             //开始批量绘制
2  EndBatchDraw();               //结束输入后再一次性刷新到屏幕
```

2.3 图形绘制

```
1  solidrectangle(x1, y1, x2, y2); //画无边框的矩形
2
3  rectangle(x1, y1, x2, y2);    //画矩形边框
4
5  line(x1, y1, x2, y2);        //画线段
6
7  setfillstyle(BS_SOLID);      //设置填充样式
8  setfillcolor(RGB(r, g, b));  //设置填充颜色
9
10 setlinecolor(RGB(r, g, b));   //设置线条颜色
```

2.4 文字绘制

```
1  outtextxy(x, y, _T("文字")); //在指定坐标写文字
2
3  drawtext(_T("文字"), &r, DT_RIGHT | DT_VCENTER | DT_SINGLELINE);
    // 在矩形区域内写文字（支持对齐、自动换行）
```

2.5 鼠标输入

```
1  MOUSEMSG m;                  // 声明鼠标消息
2  while (MouseHit())            // 检查是否有消息
3  {
4      m = GetMouseMsg();         // 取出一条消息
5      m.x, m.y                   // 鼠标坐标
6      m.uMsg == WM_LBUTTONDOWN  // 左键按下
```



```

7      m.uMsg == WM_MOUSEWHEEL      // 滚轮滚动
8      m.wheel                      // 滚轮增量
9  }

```

3 分析代码中逻辑错误及解决方法

3.1 运算符无法识别

```

1  double a, b;
2  if (ZFC2[i] == '+')      { a = p.top(); p.pop(); b = p.top(); p.
    pop(); p.push(a + b); }
3  if (ZFC2[i] == '-')      { a = p.top(); p.pop(); b = p.top(); p.
    pop(); p.push(b - a); }
4  if (ZFC2[i] == '*')      { a = p.top(); p.pop(); b = p.top(); p.
    pop(); p.push(a*b); }
5  if (ZFC2[i] == '/')      { a = p.top(); p.pop(); b = p.top(); p.
    pop(); p.push(b / a); }

```

- 在按钮处理逻辑中（第 125-161 行），乘法和除法使用的是中文字符“x”和“÷”，但在计算逻辑中（第 452-455 行），只处理标准的“*”和“/”符号
- calculate(函数只识别标准的 ASCII 运算符 (*、/)。当遇到中文字符“×”或“÷”时，它们不会被正确处理

修改：点击的时候改成点击“*”和“/”

```

// 乘法按钮 - 使用标准的 '*' 而不是 'x'
else if (j == 1 && i == 3 && ZFC[D] != '.')
{
    _tcscat_s(ZFC1, ZFC), D1 += D;
    if (D != -1)
        ZFC1[++D1] = '*'; // 使用标准乘号
    else
        ZFC1[D1] = '*';
    D = -1, ZFC1[++D1] = '\0';
    xsd = false;
    memset(ZFC, 0, sizeof(ZFC));
}

```

3.2 除法错误

```
1  if(whatnumber==4){
2      sum2=sum1/(double)sum;
3  }
```

- 未检查除数是否为零，易造成答案错误。
- 可能导致程序崩溃或未定义行为。

修改：

```
1  if (x1 == 0) {
2      res = _T("Divisor_error!");
3      formula.clear();
4      expr.clear();
5      return;
6  }
```

3.3 运算符优先级错误

```
1  while (top() == '-' || top() == '*' || top() == '/' || top() == '
    '+' && top() != '(')
```

- 因为与运算的优先级比或运算高，所以这个表达式中就是判断加号和判断栈顶是否为左括号先做与运算，再和其他判断做或运算。但实际上应该是判断加减乘除四个运算和判断栈顶是否为左括号做与运算，括号位置不对。
- 会造成结果的错误和死循环。如 $1 + 2 * 3$ 会算得 24。

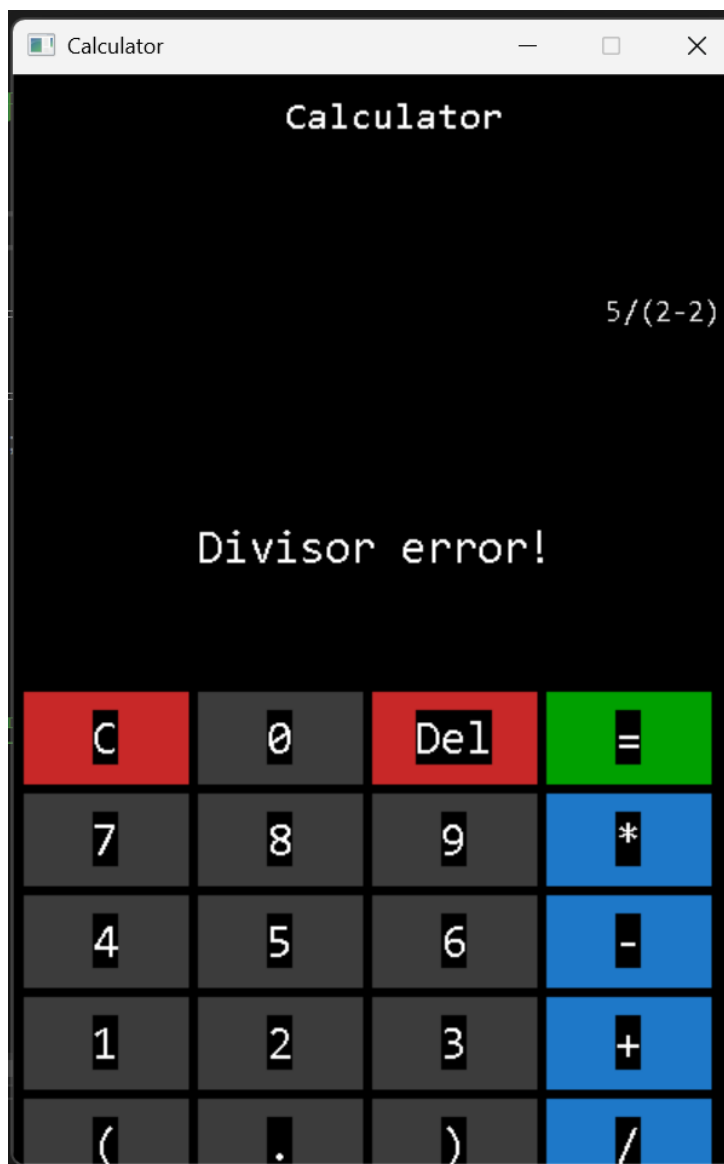
修改：

```
1  while (top() == '-' || top() == '*' || top() == '/' || (top() ==
    '+' && top() != '('))
```

4 可分析的错误原因

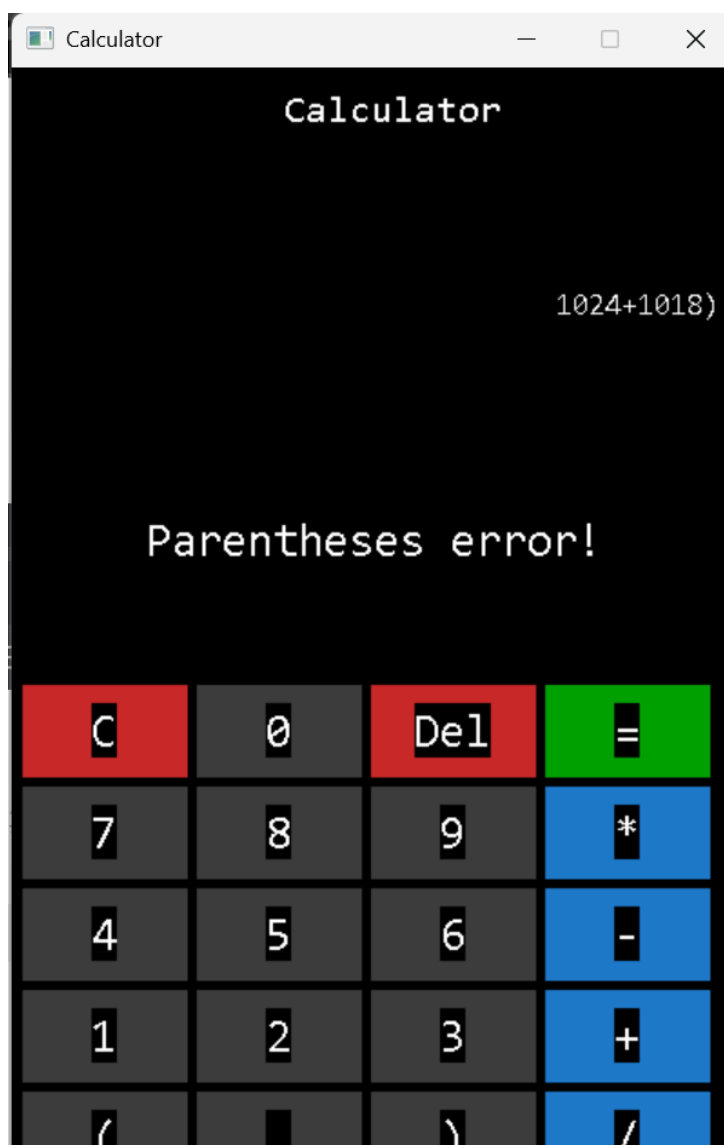
4.1 除零报错

- 示例： $5/(2-2)$



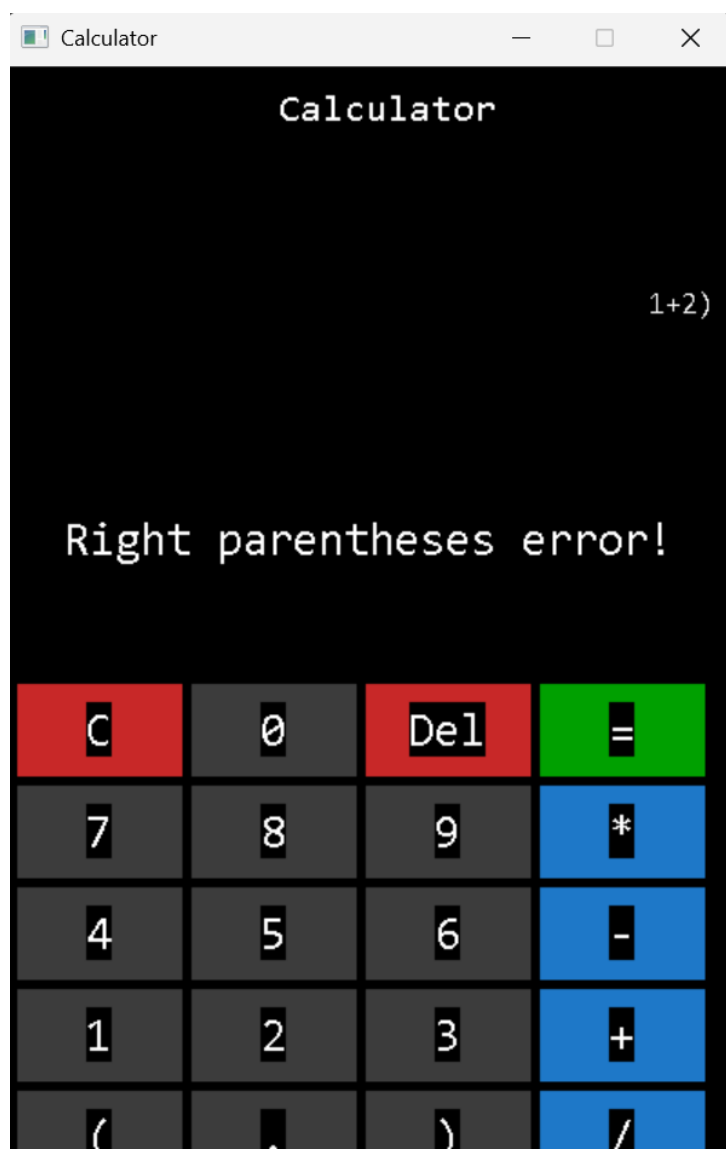
4.2 运算符和参与运算的数字无法形成运算关系

- 示例: $1024 + 1018)$

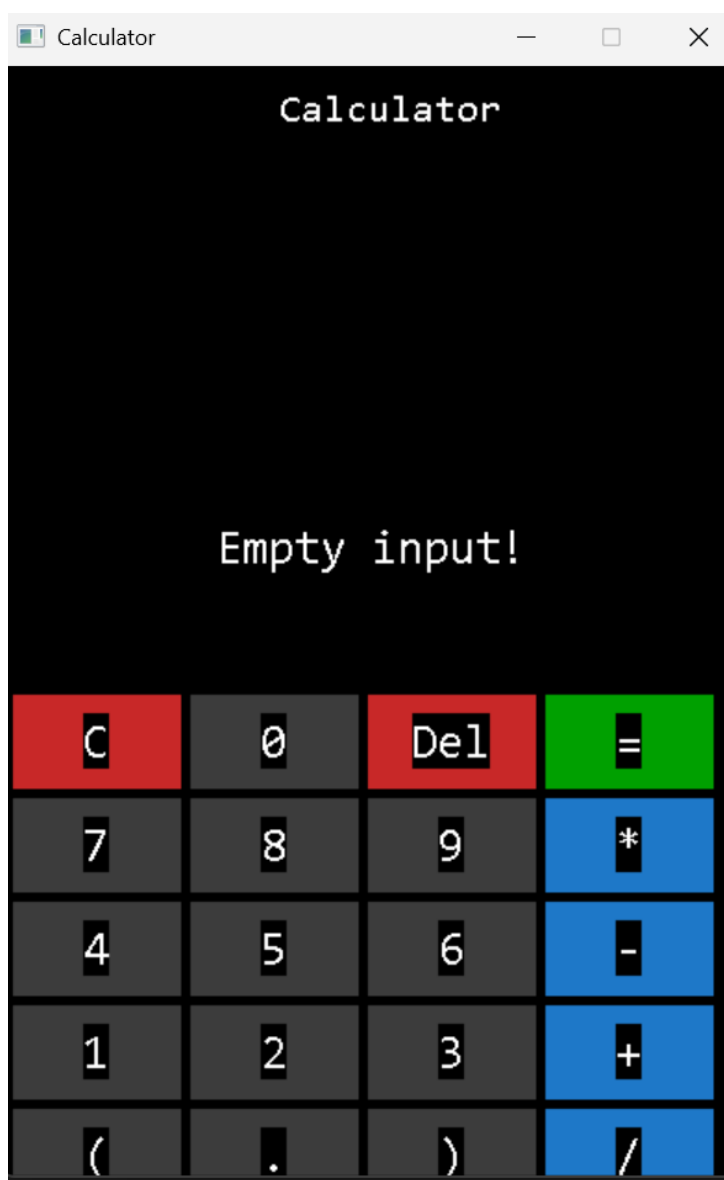


4.3 右括号多余

- 示例: $1 + 2)$



4.4 空白输入



5 实现过程中遇到的问题和解决方法

5.1 数字解析错误

- 程序原代码直接将数字字符的 ASCII 码值压入运算栈，未对连续数字、小数进行完整解析，导致无法识别多位数（如 123 被拆分为 1、2、3）和小数（如 12.34 无法正常解析），最终四则运算结果完全错误，无法实现正确的大数计算。

5.2 混淆了 void 和不带 void 函数使用

- 混淆了 C++ 函数声明与函数调用的语法，使用 `void showscreen();` 声明语句替代了 `showscreen();` 调用语句，导致界面绘制逻辑未执行

```

std::stack<double>nums;
for (int i = 0; i < newformula.length(); ++i) {
    //遇到数字时继续往后读取完整的数字
    if (newformula[i] >= '0' && newformula[i] <= '9') {
        tstring numstr;
        while (i < newformula.length() && (newformula[i] >= '0' && newformula[i] <= '9' || newformula[i] == '.')) {
            numstr += newformula[i];
            i++;
        }
        i--; //回退一位
        //把字符串转换成double
        double num = _wtof(numstr.c_str());
        nums.push(num);
    }
}

```

5.3 没把绘制函数放到 while 循环里

```

while (true) {
    updatewithinput();
    showscreen();
    Sleep(20);
}

```

导致按钮一直画不出来

5.4 支持连续运算

- expr 是界面上数字主显示区的核心变量，showscreen() 函数会优先显示 expr 的内容。这行把计算结果赋值给 expr，一方面让结果直接显示在屏幕上，另一方面支持你于这个结果继续做连续运算（比如算完 1+2=3，直接按 *5 就能接着算 3*5）。

```

double result = nums.top();
res = std::to_wstring(result);
formula.clear();
expr = res;
}

```

- 之前直接 expr.clear() 就导致无法看到之前运算的算式，可读性差。

5.5 中缀转后缀时运算式子中剩余运算符引发报错

- 表达式遍历完成后，运算符栈中残留的未处理运算符未被添加到后缀表达式中，导致生成的后缀表达式结构不完整，四则运算结果错误，甚至触发非法表达式校验报错。
- 因为只有遇到下一个新运算符时，才会把之前压在栈里的旧运算符拿出来。但算式的最后一个运算符，后面没有新的运算符了，永远不会触发弹出动作，就会一直留在栈里，导致最终的后缀表达式少了这个运算符，算式不完整，计算结果必然出错。

```
while (!ops.empty()) {  
    newformula += ops.top();  
    newformula += ' ';  
    ops.pop();  
}
```

- 因此要手动循环清空运算符栈，才能保证表达式正确。

6 代码拷贝

6.1 main.cpp

```
1 #include <windows.h>  
2 #include "easyx.h"  
3 #include "calculator.h"  
4 #include "display.h"  
5 #include "input.h"  
6 #include "calc.h"  
7  
8 int main() {  
9     initgraph(400, 600);  
10    SetWindowText(GetHwnd(), _T("Calculator"));  
11  
12    while (true) {
```



```
13         updatewithinput();
14         showscreen();
15         Sleep(20);
16     }
17
18     closegraph();
19     return 0;
20 }
```

6.2 calculator.h

```
1  #pragma once
2  #include <easyx.h>
3  #include <string>
4
5  #ifdef UNICODE
6      typedef std::wstring tstring;
7  #endif
8
9      extern tstring expr;
10     extern tstring formula;
11     extern tstring res;
12     extern bool button[4][5];
13
14     extern const int WIN_W;    // 窗口宽度
15     extern const int WIN_H;    // 窗口高度
16     extern const int BTN_COLS; // 按钮列数
17     extern const int BTN_ROWS; // 按钮行数
18     extern const int BTN_W;    // 按钮宽度
19     extern const int BTN_H;    // 按钮高度
20     extern const int BTN_GAP;  // 按钮间距
21     extern const int BTN_START_X; // 按钮起始X
22     extern const int BTN_START_Y; // 按钮起始Y
23     extern const char btnChar[4][5];
24
25     void Digit(char c);
26     void Operator(char op);
27     void leftParen();
28     void rightParen();
```

```
29     void Equal();
30     void Clear();
31     void Delete();
32
33     void updatewithinput();    // 处理鼠标输入
34     void showscreen();        // 绘制界面
```

6.3 calc.h

```
1 #pragma once
2
3 void Digit(char c);
4 void Operator(char op);
5 void leftParen();
6 void rightParen();
7 void Equal();
8 void Clear();
9 void Delete();
```

6.4 display.h

```
1 #pragma once
2 extern const char btnChar[4][5];
3 void showscreen();
```

6.5 transparent.h

```
1 #pragma once
2 #include <easyx.h>
3
4 void my_putimage(int dstX, int dstY, IMAGE *pimg,
5                 int avoid_color, int deviation,
6                 double light, int tp);
```

6.6 image.h

```
1 #pragma once
2 #include <easyx.h>
3
4 IMAGE setimage(int width, int height, COLORREF rgb);
```

6.7 input.h

```
1 #pragma once
2 void updatewithinput();
```

6.8 calculator.cpp

```
1 #define _CRT_SECURE_NO_WARNINGS
2 #include "calculator.h"
3 #include <sstream>
4 #include <iomanip>
5 #include <cmath>
6 #include <stack>
7 using namespace std;
8
9 //====global variables====
10
11 tstring expr;
12 tstring formula;
13 tstring res;
14
15 bool button[4][5] = { false };
16 const int WIN_W = 400;           // 窗口宽度
17 const int WIN_H = 640;           // 窗口高度
18 const int BTN_COLS = 4;          // 按钮有4列
19 const int BTN_ROWS = 5;          // 按钮有5行
20 const int BTN_W = 90;            // 每个按钮宽90像素
21 const int BTN_H = 50;            // 每个按钮高50像素
22 const int BTN_GAP = 8;           // 按钮之间间距8像素
23 const int BTN_START_X = 6;       // 第一个按钮的X坐标
24 const int BTN_START_Y = 300;     // 第一个按钮的Y坐标
25
26
```

```

27 const char btnChar[4][5] = {
28     {'C', '7', '4', '1', '('},
29     {'0', '8', '5', '2', '.'},
30     {'D', '9', '6', '3', ')'},
31     {'=', '*', '-', '+', '/'}
32 };

```

6.9 input.cpp

```

1  #define _CRT_SECURE_NO_WARNINGS
2  #include "calculator.h"
3  #include "input.h"
4  #include "calc.h"
5  using namespace std;
6
7  void updatewithinput()
8  {
9      ExMessage m;
10     while (peekmessage(&m, EX_MOUSE))
11     {
12         for (int i = 0; i < BTN_COLS; i++)
13         {
14             for (int j = 0; j < BTN_ROWS; j++)
15             {
16                 int bx = BTN_START_X + i * (BTN_W + BTN_GAP);
17                 int by = BTN_START_Y + j * (BTN_H + BTN_GAP);
18
19                 if (m.x >= bx && m.x <= bx + BTN_W &&
20                     m.y >= by && m.y <= by + BTN_H)
21                 {
22                     button[i][j] = true;
23                     if (m.message == WM_LBUTTONDOWN)    // 左键按下
24                     {
25                         char c = btnChar[i][j];        // 获取按钮上
26                                                         的字符
27                         switch (c)                      // 根据字符调
28                                                         用对应函数
29                         {
30                             case 'C': Clear();         break; // 清空

```

```

29         case 'D': Delete();      break;    // 退格
30         case '=': Equal();       break;    // 等号 (还
           没实现)
31         case '(': leftParen();   break;    // 左括号
32         case ')': rightParen();  break;    // 右括号
33         case '.': Digit('.');    break;    // 小数点
34         case '+': Operator('+'); break;    // 加
35         case '-': Operator('-'); break;    // 减
36         case '*': Operator('*'); break;    // 乘
37         case '/': Operator('/'); break;    // 除
38         default:
39             if (c >= '0' && c <= '9')
40                 Digit(c);          // 数字
41             break;
42     }
43 }
44 }
45 else
46 {
47     button[i][j] = false;
48 }
49 }
50 }
51 }
52 }

```

6.10 calc.cpp

```

1  #include "calculator.h"
2  #include <stack>
3
4  void Digit(char c) {
5      if (c == '.') {
6          if (expr.find('.') == tstring::npos)
7              expr += (TCHAR)c;
8      } else {
9          expr += (TCHAR)c;
10     }
11 }

```

```
12
13 void Operator(char op) {
14     if (!expr.empty()) {
15         formula += expr;
16         expr.clear();
17     }
18     formula += (TCHAR)op;
19 }
20
21 void leftParen() {
22     if (!expr.empty()) {
23         formula += expr;
24         expr.clear();
25     }
26     formula += (TCHAR)'(';
27 }
28
29 void rightParen() {
30     if (!expr.empty()) {
31         formula += expr;
32         expr.clear();
33     }
34     formula += (TCHAR)')';
35 }
36
37
38 void Clear() {
39     expr.clear();
40     formula.clear();
41     res.clear();
42 }
43
44 void Delete() {
45     if (!expr.empty()) {
46         expr.pop_back();
47     }
48     else if (!formula.empty()) {
49         formula.pop_back();
50     }
51 }
```

```
52 int priority(char ch) {
53     if (ch == '+' || ch == '-') return 1;
54     else if (ch == '*' || ch == '/') return 2;
55     else return 0;
56 }
57 void Equal() {
58     tstring full = formula + expr;
59
60     if (full.empty()) {
61         res = _T("Empty input!");
62         formula.clear();
63         expr.clear();
64         return;
65     }
66
67     std::stack<char> ops;
68     tstring newformula;
69     for (int i = 0; i < full.length(); ++i) {
70         TCHAR ch = full[i];
71
72         // 数字
73         if ((ch >= '0' && ch <= '9') || ch == '.') {
74             while (i + 1 < full.length() && ((full[i+1] >= '0' &&
75                 full[i+1] <= '9') || full[i+1] == '.')) {
76                 newformula += full[i];
77                 i++;
78             }
79             newformula += full[i];
80             newformula += ' ';
81         }
82         // 左括号
83         else if (ch == '(') {
84             ops.push('(');
85         }
86         // 右括号
87         else if (ch == ')') {
88             while (!ops.empty() && ops.top() != '(') {
89                 newformula += ops.top();
90                 ops.pop();
```

```
91         newformula += '␣';
92     }
93     if (ops.empty()) {
94         res = _T("Parentheses␣error!");
95         formula.clear();
96         expr.clear();
97         return;
98     }
99     else ops.pop();
100 }
101
102 else {
103     while (!ops.empty() && priority(ops.top()) >= priority(ch
104         )) {
105         newformula += ops.top();
106         newformula += '␣';
107         ops.pop();
108     }
109     ops.push(ch);
110 }
111
112 while (!ops.empty()) {
113     newformula += ops.top();
114     newformula += '␣';
115     ops.pop();
116 }
117
118 std::stack<double>nums;
119 for (int i = 0; i < newformula.length(); ++i) {
120     //遇到数字时继续往后读取完整的数字
121     if (newformula[i] >= '0' && newformula[i] <= '9') {
122         tstring numstr;
123         while (i < newformula.length() && (newformula[i] >= '0' &&
124             newformula[i] <= '9' || newformula[i] == '.')) {
125             numstr += newformula[i];
126             i++;
127         }
128         i--; //回退一位
129         //把字符串转换成double
```



```
129         double num = _wtof(numstr.c_str());
130         nums.push(num);
131     }
132     else if (newformula[i] == '□') continue;
133     else {
134         //检查 一个符号至少有两个数
135         if (nums.size() < 2) {
136             res = _T("Invalid□expression");
137             formula.clear();
138             expr.clear();
139             return;
140         }
141
142         if (newformula[i] == '+') {
143             double x1 = nums.top();
144             nums.pop();
145             double x2 = nums.top();
146             double x3 = x1 + x2;
147             nums.push(x3);
148         }
149
150         else if (newformula[i] == '*') {
151             double x1 = nums.top();
152             nums.pop();
153             double x2 = nums.top();
154             double x3 = x1 * x2;
155             nums.push(x3);
156         }
157
158         else if (newformula[i] == '-') {
159             double x1 = nums.top();
160             nums.pop();
161             double x2 = nums.top();
162             double x3 = x2-x1;
163             nums.push(x3);
164         }
165
166         else if (newformula[i] == '/') {
167             double x1 = nums.top();
```

```
169         if (x1 == 0) {
170             res = _T("Divisor_error!");
171             formula.clear();
172             expr.clear();
173             return;
174         }
175
176         nums.pop();
177         double x2 = nums.top();
178         double x3 = x2 / x1;
179         nums.push(x3);
180     }
181 }
182 }
183
184 double result = nums.top();
185 res = std::to_wstring(result);
186 formula.clear();
187 expr = res;
188 }
```

6.11 dispaly.cpp

```
1  #include "calculator.h"
2  #include "display.h"
3  using namespace std;
4
5  void showscreen()
6  {
7      BeginBatchDraw();
8
9      setfillcolor(RGB(0, 0, 0));
10     solidrectangle(0, 0, WIN_W, WIN_H);
11
12     settextcolor(WHITE);
13     settextstyle(25, 0, _T("Consolas"));
14     outtextxy(150, 10, _T("Calculator"));
15
16     settextstyle(20, 0, _T("Consolas"));
```

```
17 RECT rExpr = { 10, 100, 390, 160 };
18 drawtext(formula.c_str(), &rExpr,
19          DT_RIGHT | DT_VCENTER | DT_SINGLELINE | DT_END_ELLIPSIS)
20
21 ;
22
23 settextstyle(40, 0, _T("Consolas"));
24 RECT rResult = { 10, 200, 390, 380 };
25 if (!expr.empty())
26     drawtext(expr.c_str(), &rResult,
27             DT_RIGHT | DT_VCENTER | DT_WORDBREAK);
28 else if (!res.empty())
29     drawtext(res.c_str(), &rResult,
30             DT_RIGHT | DT_VCENTER | DT_WORDBREAK);
31 else
32     drawtext(_T("0"), &rResult,
33             DT_RIGHT | DT_VCENTER | DT_SINGLELINE);
34
35 for (int i = 0; i < BTN_COLS; i++)
36 {
37     for (int j = 0; j < BTN_ROWS; j++)
38     {
39         int bx = BTN_START_X + i * (BTN_W + BTN_GAP);
40         int by = BTN_START_Y + j * (BTN_H + BTN_GAP);
41         char c = btnChar[i][j];
42
43         COLORREF bgColor;
44         if (c == 'C' || c == 'D')
45             bgColor = RGB(200, 40, 40);
46         else if (c == '=')
47             bgColor = RGB(0, 160, 0);
48         else if (c == '+' || c == '-' || c == '*' || c == '/')
49             bgColor = RGB(30, 120, 200);
50         else
51             bgColor = RGB(60, 60, 60);
52
53         setfillcolor(bgColor);
54         solidrectangle(bx, by, bx + BTN_W, by + BTN_H);
55
56         settextcolor(WHITE);
57         settextstyle(30, 0, _T("Consolas"));
```

```

56         tstring label;
57         if (c == 'D') label = _T("Del");
58         else { label += (TCHAR)c; }
59         RECT rBtn = { bx, by, bx + BTN_W, by + BTN_H };
60         drawtext(label.c_str(), &rBtn,
61                 DT_CENTER | DT_VCENTER | DT_SINGLELINE);
62     }
63 }
64
65 EndBatchDraw();
66 }

```

6.12 loadimage.cpp

```

1  #include "loadimage.h"
2
3  IMAGE setimage(int width, int height, COLORREF rgb)
4  {
5      IMAGE img(width, height);
6      SetWorkingImage(&img);
7      setfillcolor(rgb);
8      solidrectangle(0, 0, width, height);
9      SetWorkingImage();
10     return img;
11 }

```

6.13 transparent.cpp

```

1  #include "transparent.h"
2
3  void my_putimage(int dstX, int dstY, IMAGE *pimg,
4                  int avoid_color, int deviation,
5                  double light, int tp)
6  {
7      IMAGE* now_working = GetWorkingImage();
8
9      IMAGE tempimg;
10     Resize(&tempimg, pimg->getwidth(), pimg->getheight());
11 }

```

```
12     SetWorkingImage(pimg);
13     getimage(&tempimg, 0, 0, pimg->getwidth(), pimg->getheight());
14
15     SetWorkingImage(now_working);
16
17     IMAGE pSrcImg;
18     getimage(&pSrcImg, dstX, dstY, pimg->getwidth(), pimg->getheight
19             ());
20
21     if (tp < 0) tp = 0;
22     if (tp > 100) tp = 100;
23
24     DWORD* bk_pMem = GetImageBuffer(&pSrcImg);
25     DWORD* pMem = GetImageBuffer(&tempimg);
26
27     int avoid_r = GetRValue(avoid_color);
28     int avoid_g = GetGValue(avoid_color);
29     int avoid_b = GetBValue(avoid_color);
30
31     for (int y = 0; y < pimg->getheight(); y++)
32     {
33         for (int x = 0; x < pimg->getwidth(); x++)
34         {
35             int num = y * pimg->getwidth() + x;
36             int R = GetRValue(pMem[num]);
37             int G = GetGValue(pMem[num]);
38             int B = GetBValue(pMem[num]);
39
40             if (abs(R - avoid_r) <= deviation &&
41                 abs(G - avoid_g) <= deviation &&
42                 abs(B - avoid_b) <= deviation)
43             {
44                 pMem[num] = bk_pMem[num];
45             }
46             else
47             {
48                 if (light > 0 && light < 1)
49                 {
50                     R = int(R * light);
51                     G = int(G * light);
```

```
51         B = int(B * light);
52     }
53     else if (light > 1)
54     {
55         R = min(int(R * light), 255);
56         G = min(int(G * light), 255);
57         B = min(int(B * light), 255);
58     }
59
60     pMem[num] = RGB(
61         (R * tp + GetRValue(bk_pMem[num]) * (100 - tp)) /
62         100,
63         (G * tp + GetGValue(bk_pMem[num]) * (100 - tp)) /
64         100,
65         (B * tp + GetBValue(bk_pMem[num]) * (100 - tp)) /
66         100
67     );
68 }
69
70 putimage(dstX, dstY, &tempimg);
71 }
```